

Testes de Engenharia do Caos com IA (O "Chaos GPT")

A Engenharia do Caos (Chaos Engineering) popularizada pela Netflix (derrubar servidores de propósito para ver se o sistema aguenta) já é conhecida. Mas e se atualizarmos isso para os dias de hoje?

- **O Conceito:** Apresente o conceito de "**Prompt Injection Testing**" e "**AI Chaos Monkey**".
- **A Atividade Prática:** Pegue um sistema simples que a turma desenvolveu ou use um formulário público. Peça para os alunos tentarem quebrar o sistema não usando *inputs* normais (como strings longas ou números negativos), mas usando engenharia social contra a camada de IA do sistema ou injetando payloads absurdos.
- **Exemplo Esdrúxulo:** O desafio é fazer o sistema aceitar um cupom de desconto chamado `ME_DA_DESCONTO_OU_O_GATINHO_CHORA` que zera o carrinho, explorando falhas de lógica de integração que os testes tradicionais de caixa-preta não pegaram.

A **Engenharia do Caos (Chaos Engineering)** é a disciplina que consiste em **mudar o foco de "esperar que as coisas quebrem" para "quebrar as coisas de propósito"** em um ambiente controlado. O objetivo é testar a resiliência de um sistema distribuído e descobrir fraquezas antes que elas causem uma falha catastrófica em produção.

A ideia central é que, em sistemas modernos e complexos (microsserviços, nuvem, APIs de terceiros), falhas não são uma possibilidade — elas são uma certeza. A Engenharia do Caos serve para garantir que, quando uma parte do sistema falhar, o todo continue funcionando (degradação suave).

O Processo da Engenharia do Caos

A prática segue o método científico tradicional, dividido em quatro etapas principais:

1. **Definir o "Estado Estável" (Steady State):** Identificar métricas que indicam que o sistema está funcionando normalmente (ex: taxa de erro abaixo de 1%, tempo de resposta de 200ms, número de vendas por minuto estável).
2. **Criar uma Hipótese:** Formular uma suposição do tipo: *"Se nós derrubarmos o microsserviço de recomendações, o usuário ainda conseguirá finalizar a compra, apenas sem ver as recomendações na tela"*.
3. **Introduzir Variáveis (O Experimento):** Injetar uma falha real no sistema. Isso pode incluir:
 - Derrubar um servidor ou container.
 - Injetar latência na rede (simular internet lenta).
 - Bloquear o acesso a um banco de dados.
 - Matar uma zona de disponibilidade inteira da nuvem.

4. **Analisar e Corrigir:** Verificar se o estado estável foi mantido. Se a hipótese falhou (ex: o sistema inteiro caiu porque o microsserviço de recomendações sumiu), você encontrou uma vulnerabilidade arquitetural que precisa ser corrigida.

O "Raio de Explosão" (Blast Radius)

Um dos conceitos mais importantes ao executar esses experimentos é o **Blast Radius**. Você nunca deve injetar caos em todo o sistema de uma vez. O teste deve começar com o menor impacto possível (ex: apenas 1% dos usuários ou em um ambiente de homologação muito fiel) e ir expandindo à medida que o sistema prova ser resiliente.

Ferramentas Atuais no Mercado

Embora a Netflix tenha começado tudo com o famoso **Chaos Monkey** (que derrubava instâncias de servidores aleatoriamente), hoje o ecossistema evoluiu muito:

- **Gremlin:** Uma plataforma completa como serviço (SaaS) para injetar falhas de CPU, memória, rede e estado de forma segura e controlada.
- **Chaos Mesh:** Uma ferramenta poderosa e *open-source* focada nativamente em ambientes Kubernetes.
- **LitmusChaos:** Outra ferramenta voltada para o ecossistema de microsserviços e práticas de Cloud-Native.
- **AWS Fault Injection Simulator (FIS):** O serviço nativo da Amazon para simular falhas em serviços hospedados na AWS.

Em suma, a Engenharia do Caos não é sobre criar bagunça; é sobre destruir o sistema de forma controlada para construir uma infraestrutura altamente confiável e antifrágil.

Primeiro, vamos desenhar o experimento (a hipótese) e, depois, ver como colocar esse "monstro" para rodar automaticamente na esteira de CI/CD (integração contínua), que é o ápice da automação de testes atuais.

1. Formulando a Hipótese (O Cenário Esdrúxulo)

Imagine que o sistema de vocês é um e-commerce de capivaras de pelúcia. O microsserviço principal é o carrinho de compras, mas ele depende de uma API de IA de terceiros para calcular o frete dinamicamente baseado no humor do cliente.

- **Estado Estável (Steady State):** O usuário consegue fechar a compra em menos de 3 segundos, e a taxa de conversão é de 98%.
- **A Falha Injetada:** A API de IA do frete vai começar a responder com **15 segundos de atraso (latência severa)** ou vai simplesmente sumir do mapa (Erro 500).
- **A Hipótese do Caos:** *"Se a API de frete falhar ou demorar, o sistema deve adotar um 'plano B' (fallback): aplicar um frete fixo de R\$ 10,00 e permitir que o cliente compre. O site NÃO pode travar com uma tela branca ou dar timeout no carrinho todo."*

2. Automatizando o Caos no CI/CD (GitHub Actions / GitLab CI)

Colocar Engenharia do Caos no CI/CD significa que **toda vez que o desenvolvedor commitar um código novo, o próprio pipeline vai tentar quebrar o sistema em homologação** antes de deixar o código ir para produção. Se o sistema "morrer" com a falha, o deploy é cancelado automaticamente.

Aqui está como estruturar esse pipeline moderno:

O Fluxo do Pipeline

[Build] → [Testes Unitários/E2E] → [Deploy em Homologação] → [DISPARAR O CAOS] → [Validar Métricas] → [Deploy em Produção]

Se usarmos uma ferramenta como o **Chaos Mesh** ou a API da **Gremlin**, o arquivo de configuração do seu pipeline (ex: GitHub Actions) ganharia uma etapa parecida com esta:

YAML

chaos-testing:

needs: deploy-homologacao

runs-on: ubuntu-latest

steps:

- name: Iniciar Ataque de Latência (Engenharia do Caos)

run: |

curl -X POST https://api.gremlin.com/v1/attacks \

-H "Authorization: Bearer \${ secrets.GREMLIN_API_KEY }" \

-d '{

"type": "latency",

"target": "api-frete-ia",

"length": "60",

"delay": "15000"

}'

Isso injeta 15 segundos (15000ms) de atraso na API por 1 minuto

- name: Rodar Teste E2E de Verificação (Cypress/Playwright)

run: |

npm run test:e2e-carrinho

O teste tenta comprar. Se o site travar por causa do frete lento, o teste falha aqui!

- name: Se o teste falhar, abortar e alertar o time

if: failure()

run: echo "O sistema não aguentou o caos! Corrijam o circuito de proteção (Circuit Breaker) antes do deploy!"

Para fechar a última aula (O Grande Gran Finale)

Mostre para os alunos o conceito de **Circuit Breaker (Disjuntor)**. Explique que, na Engenharia do Caos, a melhor ferramenta de "teste" não é um assert, mas sim a arquitetura do código.

Se o microsserviço de frete falhar, o "disjuntor" do código abre, isola a API problemática e assume o controle com um valor padrão. O pipeline passa, o caos é dominado e o cliente continua comprando.

Frase de impacto para o quadro: *"Em 2026, testar não é apenas garantir que o código funciona quando tudo está perfeito. Testar é garantir que o sistema sobrevive quando o mundo ao redor dele está pegando fogo."*